KI/SZZPP Aplikovaná informatika: povinný základ

1. Programování: funkce a cykly

Přehled požadovaných znalostí a dovedností

- literály základních tříd (*int*, *float*, *bool*, *string*), metody/operace základních tříd
- import balíčků, základní balíčky (*math*, *random*)
- proměnné, přiřazení
- podmíněné příkazy (*if*:*elif*:*else*)
- interpolace řetězců se základním formátováním (f"")
- **cykly (*for*, *while*)**
- **vlastní funkce (poziční a pojmenované parametry, příkaz *return*)**
- vyvolávání výjimek
- vstup a výstup na konzoli (*print*, *input*)

Obecná charakteristika zkušební úlohy

1. najít chybu/chyby v poskytnutém kódu v jazyce Python a opravit ji
2. podle zadání rozšířit funkčnost či poskytnout alternativní funkčnost

Ukázková úloha

V následující funkci nalezněte syntaktické i sémantické chyby a opravte je.

```
def test_of_parantheses(text: str) -> bool:
    """
       Testuje, zda jsou kulaté závorky správně uzávorkované.
      Příklad chybného uzávorkování: ")(())("
    """
    para_count = 0
    for c in text:
        if text == "(":
            para_count += 1
        elif text == ")":
            para_count -= 1
    return para_count == 0
```

Program rozšiřte o následující funkčnost:

- dodatečný poziční parametr typu *n*-tice určující dvojici znaků, sloužící jako otevírací a uzavírací závorka
- testování vstupní podmínky (řetězec musí být neprázdný)
- funkce vracející maximální úroveň zanoření závorek (pokud nejsou správně uzávorkované, vyhazuje výjimku *ValueError*)

výstup:

- opravený program
- výsledky ladění (co funguje a co nikoliv a proč?)

Seznam dostupných materiálů a technologií

- překladač Pythonu
- vývojové prostředí (*Jupyter notebook*)
- tahák Python

2. Programování: kolekce

Přehled požadovaných znalostí a dovedností

- literály základních tříd (*int*, *float*, *bool*, *string*), metody/operace základních tříd
- import balíčků, základní balíčky (*math*, *random*)
- proměnné, přiřazení
- podmíněné příkazy (*if*:*elif*:*else*)
- interpolace řetězců se základním formátováním (f"")
- **cykly (*for*, *while*)**
- **základní kolekce (seznamy, slovník) a jejich metody a literály**
- vyvolávání výjimek
- vstup a výstup na konzoli (*print*, *input*)

Obecná charakteristika zkušební úlohy

1. najít chybu/chyby v poskytnutém kódu v jazyce Python a opravit ji
2. podle zadání rozšířit funkčnost či poskytnout alternativní funkčnost

Ukázková úloha

V následující funkci nalezněte syntaktické i sémantické chyby a opravte je.

```
# kód má ke každé položce modifikovatelné sekvence s celočíselnými
prvky
# přičíst 1 je-li položka sudá

data = range(1, 10)

print(data)

for item in data:
    if item % 2 == 1:
        item += 1

print(data)
```

Program modifikujte/rozšiřte o následující funkčnost:

- inkrementován bude prvek s lichým indexem (druhý, čtvrtý, šestý …)
- místo modifikace bude vrácen nový seznam
- kontrola, zda jsou prvky skutečně čísla, pokud nikoliv vyhození výjimky

výstup:

- opravený program
- výsledky ladění (co funguje a co nikoliv a proč?)

Seznam dostupných materiálů a technologií

- překladač Pythonu
- vývojové prostředí (*Jupyter notebook*)
- tahák Python

3. Programování: základy OOP

Přehled požadovaných znalostí a dovedností

- literály základních tříd (*int*, *float*, *bool*, *string*), metody/operace základních tříd
- import balíčků, základní balíčky (*math*, *random*)
- proměnné, přiřazení
- podmíněné příkazy (*if:elif:else*)
- interpolace řetězců se základním formátováním (f"")
- **vlastní jednoduché třídy (konstruktor, metody, speciální metoda *__str/repr__*. *__contains__*)**
- vyvolávání výjimek
- vstup a výstup na konzoli (*print*, *input*)

Obecná charakteristika zkušební úlohy

1. najít chybu/chyby v poskytnutém kódu v jazyce Python a opravit ji
2. podle zadání rozšířit funkčnost či poskytnout alternativní funkčnost

Ukázková úloha

V následující funkci nalezněte syntaktické i sémantické chyby a opravte je.

```
class Semaphore:
    colors = ["red", "yellow", "green"]
    def __init__(color:str):
        self.color = color

    def __str__():
        return self.color

    def nextColor():
        """
        vrací semafor s následující barvou v sekvenci přepínání světel
        """
        return Semaphore(colors[colors.index(self.color)+1])
```

Program modifikujte/rozšiřte o následující funkčnost:

- v konstruktoru zkontrolujte, zda je předaný řetězec platný
- definujte vlastnost (*property*) ***stop***, zde je nutno při daném stavu zastavit
- doplňte metodu, která porovná (*equality*) dva semafory
- přepínání barev implementujte jako iterátor

výstup:

- opravený program
- výsledky ladění (co funguje a co nikoliv a proč?)

Seznam dostupných materiálů a technologií

- překladač Pythonu
- vývojové prostředí (*Jupyter notebook*)
- tahák Python